Ingenieria de Software II

Jaime Nazar Anchorena

Contents

Cheatsheet **5**
- Base de datos . . . 5
  - NoSQL . . . 5
  - SQL OLTP . . . 5
  - SQL OLAP . . . 5
  - Cache . . . 5
  - Cassandra . . . 5
  - MongoDB . . . 5
  - Objetos . . . 5
- Esquemas de base de datos . . . 5
  - Primario-Secundario . . . 5
  - Primario-Primario . . . 5
- Hosting . . . 6
  - Cloud . . . 6
  - On premise . . . 6
  - Salas cofre . . . 6
- Seguridad y conexiones . . . 6
  - HTTPS . . . 6
  - JWT . . . 6
  - WAF . . . 6
  - VPN . . . 6
  - Websockets . . . 6
  - Webhooks . . . 6
- Autenticacion . . . 7
  - 2FA . . . 7
  - MFA . . . 7
- Multiples instancias de backend . . . 7
- Problemas de calidad y su solucion . . . 7
  - Availability . . . 7
  - Reliability . . . 7
  - Security . . . 7
  - Performance . . . 7
  - Precision . . . 8
  - Scalability . . . 8

Interoperabilidad . . . 8
Accessibility . . . 8
Portability . . . 8
Mantainability . . . 9
Usability . . . 9
Tradeoffs Comunes . . . 9
Riesgos comunes . . . 9
Supuestos comunes . . . 9
Arbol de utilidad . . . 10
Security . . . 10
Interoperability . . . 10
Scalability . . . 10
Accessibility . . . 10
Testability . . . 10
Performance . . . 10
Availability . . . 10

Clase 1: Aclaraciones **10**

Unidad 1: Introduccion a la Arquitectura de Software **11**
Definicion: Ingenieria de Software . . . 11
Definicion: Estilo arquitectonico . . . 11
El rol del arquitecto . . . 11
El cliente . . . 12
Tiene una idea de la solución . . . 12
Tiene un presupuesto . . . 12
Tiene restricciones . . . 12
Planificacion previa . . . 12
Qué . . . 12
Cómo . . . 13
Cuándo . . . 13
Calidad . . . 13
Atributos de calidad . . . 13
Armado de la solucion . . . 14
ABC - Influencias . . . 14
Stakeholders . . . 14
Negocio . . . 14
Estructura organizacional . . . 14
Experiencia propia . . . 14
Ambiente tecnologico . . . 14
ABC - Fin de ciclo . . . 15

Clase 2: Construccion de Arquitecturas de Software **15**
ADD . . . 15
Estilos arquitectonicos . . . 16
Dataflow . . . 16

Batch Secuencial . . . 16
Pipes & Filters . . . 16
Hierarchical Layers . . . 16
Sistemas distribuidos . . . 16
Broker . . . 16
Publish-Subcribe . . . 17
Forwarder-Receiver . . . 17
Client-Dispatcher-Server . . . 17
Sistemas interactivos . . . 17
MVC . . . 17
Sistemas basados en eventos . . . 17
. . . 18

Unidad 3: Documentacion de Arquitecturas de Software 18
Modelo 4+1 . . . 18
El +1 . . . 19
Consideraciones . . . 19
C4 . . . 19
Context . . . 19
Containers . . . 19
Components . . . 19
Code . . . 19
C4 vs 4+1 . . . 19

Unidad 4: Cuando se disena una arquitectura 20
Architectuural guaardrails . . . 20
IDP . . . 20
Evaluacion . . . 21
Arbol de utilidad . . . 21
Lightweight ATAM . . . 21

Unidad 5: Persistencia 21
Archivos . . . 22
Prevalencia . . . 22
Ventajas . . . 22
Desventajas . . . 22
Bases de datos . . . 23
Bases de datos relacionales transaccionales(OLTP) . . . 23
Bases de datos multidimensionales(OLAP) . . . 23
Bases de datos de objetos . . . 23
Mapeo objeto relacional(ORM) . . . 23
Impedancia objeto-relacion . . . 24
Escabilidad de bases de datos relacionales . . . 24
Primario - Secundario . . . 24
Primario - Primario . . . 24
Teorema CAP . . . 24

Sharding 25
Tablas como columnas 25
JSOM blobs 25
Bases de datos no relacionales 25
Columnares 25
Clave/Valor 25
Documentos 26
Ahora bien, como eligo? 26

Cheatsheet

Base de datos

NoSQL

Para una cantidad masiva de datos.

SQL OLTP

Es CA, util para funciones de agregacion.

SQL OLAP

Para uso analitico(reporting), siempre agregando y no eliminando datos.

Cache

Aumenta tiempo de respuestas, pero muy caras.

Cassandra

Columnar, permite cantidades masivas de lectura y escritura. Corre en cluster, mantiene comunicacion entre nodos con Gossip, no sirve cuando se requiere transaccionalidad.

MongoDB

Base de datos NoSQL, no brinda alta disponibilidad. Sirve cuando se requiere flexibilidad y proporciona alto volumen de escrituras.

Objetos

Base de datos NoSQL, guarda objetos complejos en cualquier lenguaje de programacion. Automatiza las operaciones mas comunes(CRUD) y facilita programacion con el tradeoff de performance y complejidad.

Esquemas de base de datos

Primario-Secundario

Busca escalar operaciones de lectura y solo se aplican las escrituras sobre el nodo primario. Si se cae el nodo primario, uno de los secundarios es elegido para tomar su rol.

Primario-Primario

Busca escalar operaciones de escritura, pero trae problemas de concurrencia y por eso no todos los motores RDBM soportan ese modo de operacion. MySQL lo tolera con Group Replication, para Postgres se requieren extensiones. Se sacrifica performance.

Hosting

Cloud

Todo el sistema se encuentra en la nube, como en AWS. Solucion mas barata y permite aumentar el alcance(expandirse a otros paises) de manera eficiente. Cortes de luz/internet ya contemplados.

On premise

Los servidores son propios de la empresa, pueden estar tanto en un datacenter compartido como en el mismo edificio. Aumenta el ownership del sistema, deben contemplarce los casos de cortes de luz/internet.

Salas cofre

Salas para almacenar servidores, similar al anterior.

Seguridad y conexiones

HTTPS

Version cifrada de HTTP

JWT

Firma digital para verificar la identidad del que manda los datos. Util cuando se tiene comunicacion con APIs.

WAF

Servicio de seguridad perimetral que analiza, filtra y bloquea el trafico HTTP/HTTPS malisioso dirigido hacia la webapp. Protege de SQL Injection, XSS, DDoS(solo de capa 7).

VPN

Conexion cifrada y segura mediante un tunel sobre infraestructura publica, permitiendo que los sistemas se comuniquen como si estuvieran en una red privada local.

Websockets

Protocolo de comunicacion bidireccional, full-duplex y persistente sobre unica conexion TCP. Permite una interaccion fluida y en tiempo real entre cliente y servidor sin la sobrecarga del modelo tradicional peticion-respuesta. Traw problemas de availability si se usa con un backend que corre en multiples instancias pues no hay forma de reestablecer la conexion si se cae un servidor del backend.

Webhooks

Mecanismo de comunicacion asincronicos basado en eventos mediando una aplicacion evia automaticamente datos en tiempo real a otra a traves de un HTTP POST. No se puede usar con una SPA.

Autenticacion

2FA

Subconjunto de MFA, exige dos evidencias de distinta categoria para validar la identidad de un usuario.

MFA

Mas general, requiere dos o mas factores de verificacion independiente de distinta categoria.

Multiples instancias de backend

Brinda escalamiento horizontal, aumentando la capacidad de computo bajo demando, aumenta disponibilidad y permite realizar procesamiento en paralelo aumentando la performance. Sin embargo, tiene el riesgo de la caida del load balancer, pero no el riesgo de la caida de una instancia del backend.

Problemas de calidad y su solucion

Availability

- Ataque DDoS -> Web Application Firewall (WAF)
- Caída Backend -> Múltiples Instancias + Load Balancer
- Caída Base de Datos -> Replicación Primaria-Secundaria si es relacional o Funcionamiento en Clúster para no relacionales.
- Corte de Internet -> Múltiples Proveedores
- Caída de la cola -> Asegurarse de especificar que corra en clúster
- Corte de luz -> Generador o doble servicio de luz
- Caída de Internet -> Doble proveedor

Reliability

- Falla la DB -> DB secundaria
- Se corta la luz -> grupo de electrogeno
- Se cae el servidor -> Escalamiento horizontal

Security

- Ataque DDoS -> Web Application Firewall
- Ataque MItM -> Conexiones HTTPS
- Ataque a operadores con info privilegiada -> Usar VPNs en sistemas internos
- Robo de credenciales -> Multi Factor Authentication (MFA)
- Gestión de Accesos -> Access Control List (ACL) en el Backend
- SQL Injections -> Sanitizar entradas en la SPA.
- Ingresos maliciosos a la BD -> Encryption at rest, la data se guarda encriptada.
- Datos muy sensibles -> Hosting privado en una sala cofre (solamente se accede con llave)

Performance

- Las lecturas tardan mucho -> Caché en el back. Importante definir qué se guarda y cuándo se borra.
- Demasiadas solicitudes -> Proceso en paralelo (sirve tener muchas instancias) y multithreading.

- Reportes Real Time -> Réplica de sólo lectura de la base de datos
- Reportes No Real Time -> Se copian los datos de la BD SQL OLTP a una OLAP mediante ETL (Extract, Transform, Load). Hacer el reporte en horarios de baja demanda.
- Caída del elemento activo -> Tener un heartbeat entre instancias para saber cuándo usar las pasivas.
- Lectura en real time -> websocket
- Búsquedas en múltiples fuentes -> procesamiento asincrónico y en paralelo con un timeout.

Precision

- Error de Punto Flotante -> punto fijo
- Sensor puede medir mal -> dos en activo-activo
- Propagación de errores en cálculos intermedios -> métodos numéricos
- Faltan mediciones -> interpolación de datos
- Mediciones varían mucho -> Intervalos de confianza
- Guardar datos como string en la base de datos
- Validar que la UI permite presentar hasta x cifras decimales

Scalability

- Picos Recurrentes -> Load balancer
- Muchos accesos a la SPA desde distintos puntos -> hosting en CDN
- Bases de datos que permitan sharding y, por ende, escalamiento horizontal.
- Crecen las lecturas -> read replicas

Interoperabilidad

- **Patrón adapter**: patrón de diseño estructural que permite la colaboración entre objetos con interfaces incompatibles, actuando como un traductor que convierte la interfaz de una clase existente en otra interfaz que el cliente espera.
- Exponer una API con requests que sólo toleren lecturas para que otros servicios puedan integrar. Fault Tolerance
- Fallo en la carga de un archivo necesario para generar un reporte -> el sistema sigue funcionando normalmente basándose en la información disponible.
- Proceso ETL(Extraction, Transformation, Load), combina, limpia y organiza datos de multiples fuentes en un unico conjunto de datos coherente(sistema batch).

Accessibility

- Probar con distintos usuarios
- Internacionalización
- Diseños HCI
- Alt en imagénes para lectores de pantalla
- Sistemas de lectores de pantalla

Portability

- PWA permite acceso desde dispositivos móviles

Mantainability

- Se implementan test continuos y automáticos para que, ante cualquier modificación, se pueda asegurar que el resto del sistema siga funcionando de manera adecuada
- El sistema es CI/CD i.e de integración y distribución continuas.
- El sistema esta modularizado permitiendo agregar cambios fácilmente, corregir errores, agregar funcionalidades, etc.
- Mediante el uso de Ionic Framework podemos crear una aplicación web y Android a partir de una única base de código.

Usability

- El sistema soporta las últimas 10 versiones de Safari, Chrome, Mozilla, etc.
- El sistema abarca todas las necesidades de todos los usuarios: se hacen pruebas empíricas para testear que cualquier tipo de usuario se sienta satisfecho a la hora de usarlo -> screen readers, speech to text, etc.
- El sistema soporta las últimas versiones de Android.
- El cliente tendrá conexión persistente con el servidor que le informará al cliente de cualquier modificación en los prestadores y/o los contratos mediante un server-push (server push vs polling).

Tradeoffs Comunes

- Availability vs Costos al tener escalamiento horizontal
- Performance vs Security al tener encryption at rest
- Precision vs Performance si se tienen operaciones de punto fijo
- Performance vs Availability si se tienen websockets, ya que si se cae un nodo, ningún otro toma la conexión y se muere.
- Priorizamos seguridad por sobre usabilidad: para mantener la seguridad del sistema, se le va pedir a los usuarios contraseñas con ciertas restricciones, two factor authentication, etc.
- Priorizamos disponibilidad por sobre costos, replicando la base de datos, instalando un generador eléctrico, duplicando el servidor.

Riesgos comunes

- No todos los usuarios tengan conexion a internet
- Debido a sharding de la BD se puede dar que algun dato quede inconsistente
- Si es on premise, riesgo de incendio, inundacion, etc.
- Se llena el queue mas rapido de lo que se procesa
- Falla el heartbeat (conexión entre pasivo-activo)
- Que se caiga el switch o el load balancer.

Supuestos comunes

- (sin scalability) suponemos que no va a escalar rapidamente y por lo tanto no es necesario que sea escalable
- (en caso de DSL por customizability) suponemos que se ponen de acuerdo en cuanto a la informacion provista y al formato en la que se presenta dicha informacion

Arbol de utilidad

Nodos intermedios como características (features) que contribuyen al atributo.

Security

- Data confidentiality
- Data integrity

Interoperability

- Third party systems

Scalability

- Overwhelming demand

Accessibility

- User experience
- Assistive technologies

Testability

- Integration tesitng
- End-to-end Indentity API

Performance

- Data latency
- Transaction throughput

Availability

- Hardware failure
- Failsafe
- COTS(comercial off-the-shelf) failure

Clase 1: Aclaraciones

- Es diferente dependiendo de la comision
- Es totalmente practica
- Hacer y consultar por mail todo lo posible
- Mediados de octubre unico parcial
- El final es teorico/practico, IMP mandar mail de como esta formado el grupo
- No se manda el enunciado del final hasta que todos manden el mail
- El cuadro de atributos es el unico que se puede tener en el examen
-

Unidad 1: Introduccion a la Arquitectura de Software

Definicion: Ingenieria de Software

La Ingeniería de Software es la rama de la ingeniería que aplica los principios de la ciencia de la computación y las matemáticas para lograr soluciones costo-efectivas a los problemas de desarrollo de software, es decir, permite elaborar consistentemente productos correctos y costo-efectivos.

Contruir un proyecto de software es muy complejos, ocupa tareas distintas y por tanto requiere de perfiles distintos(que podrian o no ser de la misma persona).

> Software architecture encompasses the set of significant decisions about the organization of a software system including: - The selection of the structural elements and their interfaces, - Behavior as specified in collaboration among those elements, - An architectural style that guides this organization."

Definicion: Estilo arquitectonico

Igual que con patrones, el **estilo arquitectonico** tambien se toma de la arquitectura. Por ejemplo, decir que un edificio es brutalista o gotico nos dice un monton de que se puede encontrar adentro, cuando se contruyo, etc. La totalidad del edificio sigue estos lineamientos, y esa es la idea que llevamos al software. Nos indica como funciona nuesto sistema como un todo.

> La coherencia interna es importante y no se puede cambiar, sobrevivira el paso del tiempo.
>
> Un sistema legacy es un sistema sumamente existoso pues perduro a pesar del paso del tiempo. Se sigue manteniendolo pues aun aporta valor.

El rol del arquitecto

Su definicion varia segun la empresa, pero son tren tipos:

- Arquitecto de soluciones(rol de negocio)
- Arquitecto de integracion(rol especifico)
- Arquitecto de software(rol operativo, el mas generico)

Vamos a tratar especialmente del arquitecto de soluciones, y un poco del de integracion en SOA. Actualemente, el rol del arquitecto esta desapareciendo, en la practica cada uno del equipo de programacion esta creando una arquitectura de software sin la necesidad de tener el rol. Sin embargo, para ello se requiere:

- **Liderazgo**: Debe ser el modelo a seguir.
- **Negocio**: No debe perder de vista que el proyecto soluciona un problema.
- **Técnico**: Desde programación a hardware.
- **Comunicación**: Debe convencer a jefes, equipo, cliente, etc.
- **Negociación**: Siempre hay compromisos.
- **Visión**: Las decisiones son hacia el futuro.

Es importante conocer el negocio, la necesidad presente y futura sobre la que tiene que operar. Hay que predecir la direccion que tomara el negocio para tener en cuenta futuros cambios.

El cliente

Al final del dia hay un problema, ya sea interno o externo de la organizacion, para el cual se necesitauna solucion. El cliente tienen una idea general y el grado de detalle suele variar mucho. Sabe lo que quiere conseguir, pero no sabe la complejidad y el costo que tiene. El cliente:

Tiene una idea de la solución

- Sabe qué quiere (a grandes rasgos)
- IWKWYSI (I will know when I see it)
- Ignora la complejidad real de la tarea

Tiene un presupuesto

- Cuanto menos, mejor

Incluso si dice que no, tiene un presupuesto y se va a bsucar lo mos barato.

Tiene restricciones

- Fecha de puesta en producción
- Tecnológicas
- Regulaciones

Planificacion previa

Se necesita **mucha** planificacion previa.

Qué

- Alcance
- Factibilidad
- Riesgos

Son los requerimientos, los de usuario(AKA casos de uso), los del sistema, los funcionales y los no funcionales. Los no funcionales no son autocontenidos, son trasversales al sistema y no tienen principio ni fin, a diferencia de los funcionales(no pueden ser un user story).

Funcionales Los funcionales son los faciles, tanto de programar como de validar. Nos vamos a concentrar en los no funcionales pues ya sabemos como hacerlos. Sin embargo, son dificiles de relevar, como se si son todos? como se si son los correctos?

Se puede hablar con el cliente, pero como sabemos si el cliente los sabe. Es la **incertidumbre**, inherente a cualquier proyecto de software, y solo desaparece cuando el proyecto se termina.

Nunca se va a poder eliminar la incertudumbre, pero hay que empezar eventualmente. Las cosas que no quedaron claras se resuelven con supuestos, los cuales deben ser:

- Posibles
- Probables

  No hay forma de que la arquitectura no falle, no es improbable

Siempre deben ser documentados, al final del dia es la definicion misma del sistema.

Cómo

- Diseño del sistema
- Recursos de hardware y software requeridos
- Equipo necesario

Cuándo

- Armar un plan de trabajo, estimar y planificar

  Esto permite además conocer el costo. El costo mas grande en un proyecto de software son los desarrolladores.

Calidad

Define si el sistema cubre todos los casos de uso, pero no alcanza con si los cumple, sino que tambien debe tener en cuenta como.

> The capability of a software product to satisfy stated and implied needs when used under specified conditions(IEEE)

La calidad tiene multiples dimensiones, las cuales se conocen como atributos de calidad. El ISO 25000 tiene una(no la unica) definicion de atributos de calidad, son muchos.

Atributos de calidad

Todos son criticos para el exito, se infieren a partir de los requerimientos no funcionales. Es importante determinar cuales son los que tenemos que poner el foco.

> En ingles, como son habilidades, terminan en -ity

No vamos a abarcar todos, sino los mas comunes(ver cuadro en apunte). Al momento de determinarlos se infieren de:

- La operación del negocio
- El marco regulatorio
- El entorno competitivo y las expectativas del usuario

  Reliability es mas duro que disponibilidad, pues se refiere a cuanto el sistema pueda perdurar en el tiempo y funcionar de manera correcta

  Testeability no es tan trivial, hay sistemas que son dificiles de probar. Por ejemplo, el envio de mails en PAW

Importante destacar, que no todos los atributos son **criticos**, hay que elegir cuales son los que realmente importa, no significa que se ignoren los demas, sino que no se pondra ningun esfuerzo ecepcional para complirlos.

Por otro lado, hay que cuantificarlos a los que elegimos, un sistema que necesite ser disponible, cuan disponible? 90%, 99%?

> No existe 100% de disponibilidad, cada 9 que le sumo incriementa el costo en un orden de magnitud

Entonces, hay que poner un limite de cuanto se quiere perfeccionar cada atributo, lo cual en si es otra decision que asume riesgos. Este tipo de desiciones debe poder ser justificadas y demostrar que se cumple. La mayor fuerza suele ser el marco regulatorio, de que se puede esperar como minimo.

Armado de la solucion

El Arquitecto debe tomar todos los requerimientos y constraints y catalizarlos en una solución. Todo esto, que no es poca cosa, no es todo lo que define una arquitectura, la arquitectura tambien es resultado de las influencias del arquitecto, de lo disponible tecnologicamente, de las buenas practicas del momento, la experiencia de la organizacion y del arquitecto(son factores internos y externos).

Pero estos no son los únicos inputs que definen la arquitectura… La arquitectura es el resultado de influencias en aspectos técnicos, sociales, culturales y de negocio.

Una arquitectura influye en aspectos técnicos, sociales, culturales y de negocio de futuras arquitecturas.

> Esto es el Architecture-Business Cycle (ABC)

ABC - Influencias

Esto tiene sus consecuencias, pues el equipo mejora sus habilidades con las tecnologias, la organizacion tiene mas sistemas a mantener, etc. Esto genera un loop de auto-refuerzo donde sera mas propensa la empresa a volver a usar esas tecnologias.

Stakeholders

Tienen expectativas y restricciones sobre el sistema, como presupuestarias, donde a veces se pueden contradecir entre si. El arquitecto debe detectarlas todas y buscar el mejor compromiso posible.

Negocio

Hay dos tipos a tener en cuenta:

- Negocio inmediato(Aprovecha sistemas existentes)
- Negocio a largo plazo(Invertir en nueva infraestructura para el futuro)

Estructura organizacional

Los conocimientos previos, que licencias hay disponibles, que herramientas hay, como trabaja el equipo.

Experiencia propia

Es raro que se vuelva a utilizar una tecnologia que dio resultados pobres en el pasado.

Ambiente tecnologico

La tecnología disponible, estándares de la industria o técnicas de ingeniería de software influye sobre ella.

> Al final del dia el objetivo fundamental es reducir el riesgo

Cuando una tecnologia que ya usamos falla es mas facil de resolver que con una tecnologia nueva, y por lo general los fallos son en produccion. Esto plantea una paradoja, pues se quiere reducir los riesgos a la vez que se quiere mantener reelevante en el mercado.

ABC - Fin de ciclo

Una vez se logra implementar una tecnologia nueva, se convierte en una decision estrategica, fue un riesgo. Por lo tanto, hay que ser cauto sobre que proyecto se utiliza para probar algo nuevo.

Encima, una vez funcione el sistema, hay que mantenerlo, se debe seguir utilizandolo. Hay que decidir cuando se va a diversificar el stack tecnologico.

Una vez que la arquitectura está creada y el sistema ha sido desarrollado, ambos afectarán:

- La estructura y objetivos de la organización que los desarrolló
- Los requerimientos futuros de los clientes
- La experiencia de los arquitectos y desarrolladores, dado que la base de experiencia corporativa ha sido mejorada
- Generan tecnología que será tenida en cuenta para ser aprovechada en futuros proyectos.

  Es un problema de gestion de riesgo

Clase 2: Construccion de Arquitecturas de Software

De los requerimientos funcionales y de los atributos de calidad se debe producir una arquitectura candidata. Es una forma de disenio llamada Attribute Driven Design. Una vez se tiene una arquitectura candidata se lleva a cabo un proceso de iteracion para perfeccionarla.

Con ADD se tiene como entrada General scenarios que se utilizan para formar Requirements. Luego se llega a una arquitectura.

ADD

0. Se da un orden de prioridad a los atributos de calidad, pues mejorar una atenta en contra de otra. Esto implica la creacion de un criterio para saber cual priorizar.
1. Determinar como va a ser la arquitectura, conceptualizarla. Usuarios, sistemas externos, donde va a ir la informacion, etc. Version mas inocente y minimalista que cumpla con los requerimientos funcionales.
2. Tomar el primer atributo y proponer escenarios que los pongan a prueba, esto permite identificar donde se rompe, los riesgos. Estos escenarios se conocen como los **drivers de arquitectura**. Hay riesgos que se pueden determinar como aceptables, aceptando que pueden pasar cosas, entonces hay que determinar como solucionar el fallo.
3. Si se cambia algo hay que re-plantearse cada escenario, volver al paso 2. Entonces, cuantos mas escenarios hay, mas se tarda en volver a analizar.
4. Fin, se obtiene una arquitectura candidata, encenarios, riesgos asumidos, trade-offs asumidos y supuestos asumidos. Sirven para la verificacion post implementacion, para que lo verifique un stakeholder y simplemente como documentacion.

   Los primeros escenarios del paso 2 son los mas probables

Por mas que haya riesgos que se asumen, hay que documentarlos y crear un plan de contingencia.

Estilos arquitectonicos

Es lo que se busca al conceptualizar el problema, son patrones de alto nivel que describen la organizacion general de un sistema en pos de ciertos atributos de calidad. Seguir un estilo ya probado provee consistencia y previsibilidad.

Dataflow

Estilo arquitectonico, externosl objetivo es procesamiento de datos, no necesariamente un sistema interactivo. Hay varios:

Batch Secuencial

Los datos se transmiten entre programas como un todo, en general mediante archivo. El procesamiente ocurre offline, no hay nada sincronico y los procesos no saben de la existencia de los demas. Se acumula informacion y en un momento dado se procesa.

Pero, si falla, se tiene que revertir todo y abortar el proceso o por ahi puede guardar el punto de procesamiento y resumir luego. Hay que documentar que hara en caso de un error.

Este estilo tiene como ventaja procesos independientes y permiten el reprocesamiento sin perdida de informacion. Sin embargo, no permite procesamiento real-time. En todo los casos documentar como manejar fallas.

Pipes & Filters

En este caso se tiene procesamiento en tiempo real, se tienen procesos chicos donde mediante pipes la salida de uno es la entrada del siguiente. Es un stream de datos, es decir, no se necesita de un fin para poder emitir la salida. Son procesos extremada mente flexible y se utiliza mucho en los OS. Pero bueno, si alguno de los procesos falla, cualquier dato que este en transito se pierde. Entonces, tiene una baja tolerancia a errores en el procesamiente. Es el pipeline clasico de arqui.

Hierarchical Layers

Cada capa provee servicios a la capa superior y es cliente de la capa inferior, el problema es encontrar el numero perfecto de capas. Las capas pueden ser logicas o fisicas. Divide un problema complejo en abstracciones, mejoras en una capa impactan en todo el sistema y suponen reusabilidad y facil intercambio de implementaciones.

Sin embargo, no todos los sistemas se pueden estructurar asi, hay un posible deterioro de performance con muchas capas(peor si son fisicas, por ejemplo, saltos entre servidores) y los cambios suelen tener que replicarse en todas las capas.

Sistemas distribuidos

Como se descubren miembros de la red y como se comunican.

Broker

Coordina comunicacion entre clientes y servidores, busca resolver el problema de encontrar un servicio y el de como comunicarse con el servicio. Los hace operando como un proxy, los servidores exponen servicio a

partir del broker. Facilita el discovery de servicios, los servicios pueden no estar expuestos en forma directa a la red, pero el broker es un Single Point Of Failure(SPOF) stateful. Si falla se cae todo.

Publish-Subcribe

Analogo al patron **Observer** en un contexto distribuido. Un componente **publisher** notifica a muchos consumidores **subscribers** suscritos a el. Hay un sistema en el medio que se encarga de hacer el broadcast a los subscribers, se conoce como el **topic**(topico). Este se encargara de hacer la multiplexacion. Ademas, estan completamente desacoplados, el sistema de origen no sabe de la existencia de los receptores y viceversa.

Entonces, los publishers y subscribers no se conocen mutuamente, solo se conocen los topics. Es posible agregar y quitar suscriptores en forma transparence. La desventaja que tiene es que requiere consideraciones especiales respecto de la perdida de eventos, segun que solucion elijo puede o no haber perdida de mensajes.

> Un webhook es muy parecido, hay implementaciones de topicos que funcionan como webhooks.
> Pero difieren en como lo implemmentas.

Forwarder-Receiver

Resuelve exclusivamente los problemas de comunicacion, se abstraen por completo y resuelven solos los problemas. Permite consumir servicios remotos como si fuesen metodos locales. Corba y Java RMI son implementaciones. Entonces, simplifica el uso al ser transparente, pero se esconde la complejida, por lo que se desconoce si tiene mucho overhead, si hay algun problema oculto, etc.

Client-Dispatcher-Server

Resuelve el problema de encontrar a los servicios, pero la conexion va a ser directa. Permite conexiones cuando el **server** no tiene una direccion fija. Por ejemplo, bit torrent, aplicaciones de mensajeria / video-conferencia.

Sistemas interactivos

Sistemas que esperan a un input, para poder reflejar el cambio en pantalla. Clasico, MVC, PAC, MVP, VIPER, etc. Conceptualmente, todas estas opciones hacen lo mismo, lo que cambia son las abstracciones y responsabilidades. El objetivo es separar la logica de negocio de las vistas de presentacion.

MVC

El mas popular, divide la aplicacion en Modelos, Vistas y Controladores. La desventaja principal es que hay un **trickle effect** entre componentes cuando cambia el modelo de datos.

Sistemas basados en eventos

Permite crear sistemas distribuidos con bajo acoplamiento. El sistema se divide en componentes completamente agnosticos uno de otros. Conceptualmente existen tres componentes:

- Event producer
- Channel
- Event consumer

Notar que es conceptual pues un componente puede ser producer y consumer al mismo tiempo. Estilos aplicables:

- Single Event Processing
- Event Stream Processing
- Complex Event Processing

- Online Event Processing

Complex event: Si un usuario hace un login y cambia password y va a hacer una compra a un articulo muy caro, son eventos normales, pero si pasan las tres en secuencia es un indicador de posible fraude. Este tipo de analisis es lo que permite hacer este estilo.

Tiene ventajas y desventajas(ver apuntes).

Existen nun monton mas de estilos, investigar a voluntad.

> Libro para investigar patrones: Patterns of Entreprise Application architecture.

Unidad 3: Documentacion de Arquitecturas de Software

Las arquitecturas no sirven si no estan bien documentadas, sirven para validar que lo construido cumple con la idea original. La gente tiene que poder ver los principales elementos funcionales, como se conectan, que caracteristicas operacionales tinene, etc. Se deja registro de todas las decisiones tomadas.

Es muy importante lograr el nivel de detalle corracto a documentar, se busca balancear:

- Vision general
- Detalle particular

Se debe crear una descripcion de la arquitectura que incluya todo. La documentacion son documentos vivos pues debe estar actualizada. El nivel de detalle debe ser el minimo necesario. El uso de patrones y practicas estandar permite decir mucho con poco. Hay muchas maneras de documentar arquitecturas, vamos aa ver una como referencia: 4+1.

La idea es tener distintas vistas del sistema para que cada una ponga foco en cosas distintas.

Modelo 4+1

- Vista logica(dominio, persistencia)
- Vista proceso(concurrencia, sincronizacion, en ejecucion)
- Vista de desarrollo(vision estatica, librerias, org. del software)
- Vista fisica(Mapeo del software con el hardware real, distribucion de hardware)

Cada vista ataca un problema en particular, “le habla” a stakeholders distintos. Van a tener distinto tipos de diagramas. Cada vista tiene un estandar UML.

El +1

Esta es la vista faltante, son los escenarios. Es la vista que cubre las otras 4, su objetivo es validar la arquitectura y explicar como los componentes se comportan en runtime.

Consideraciones

- El modelo se adecua a cada paso
- Proceso iterarivo
- Complejidad e interrelacion

Es posible elegir y eliminar vistas, si la arquitectura cambia hay que mantener actualizada la documentaccion. Importante no tener inconsistencias, pues las vistas estan muy relacionadas, entonces puede ocurrir que haya contradicciones y esto esta mal.

C4

Inspirado en 4+1, surge como alternativa C4. Compuesto por:

- Context
- Containers
- Components
- Code

No es que tiene otras maneras de ver la arquitectura, sino que van del nivel mas alto de abstraccion al mas bajo. Es independiente de la notacion, lo importante es que las decisiones se entiendan facil.

Context

El sistema es una caja negra, lo que importa es documentar con que sistemas externos se conecta.

Containers

Explora que hay adentro de la caja negra, busca las aplicaciones independiantes. Por ejemplo, API, Redis, DB, etc. Son cosas que vamos a poder deployar a un servidor.

Components

Explora lo que hay adentro de los contenedores, pueden aparecer librerias, estructuras de codigo, etc. Como va a estar construido el contenedor.

Code

Se baja a implementacion, diagramas de clase, EDRs, etc.

C4 vs 4+1

El concepto de tailoring aplica por igual, muchos arquitectos usan un approach hibrido.

Unidad 4: Cuando se disena una arquitectura

Hay que entender en que momento se realiza una arquitectura, de donde sale. Hay dos grandes opciones:

- BDUF(Big Design Up Front)
- YAGNI(You Are Not Gonna Need It)

BDUF requiere de una inversion mas significativa en estapas iniciales. YAGNI se va adaptando a medida que avanza el proyecto, involucra realizar refactors cada tanto, lo que implica costos. El costo aumenta a medida que mas se realizan desiciones, por lo que es importante tomar decisiones con la mayor informacion posible.

Ambas plantean un tradeoff de cuanto se quiere invertir upfront vs que tan flexible se quiere que sea el sistema. Un paso intermedio es JEDUF(Just Enough Design Upfront), que ayuda en los casos que tenga que tirar todo y empezar de nuevo mientras.

Para los proyectos pequenos, usar YAGNI es mas eficiente. Mientras que para proyectos grandes usar BDUF suele ser mas favorecedor pues hay una arquitectura de fondo.

Architectuural guaardrails

Las organizaciones modernas, valoran mucho la velocidad y autonomia de los equipos, que puedan armar sus propias soluciones. Por lo tanto, no suele haber rol de arquitecto. Sin embargo, establecen limites dentro de los cuales deben operar. Estos limites son los **Arquitectural Guardrails**.

Son lineamientos general que setean expectativas y limitan opciones. Se busca un balance entre autonomia y el nivel de predictividad que tiene cada equipo.

Ejemplos:

- Todo el procesamiento se hace sobre AWS
- Todo servicio debe responder en menos de 50ms
- Todo servicio debe programarse en Go o Kotlin

IDP

Internal Development Platform, son internas a cada organizacion, son mecanismos para automatizar y enforzar los Arquitectural Guardrails. No te dan una cuenta AWS, sino que si o si se usan estas herramientas.

Todas ofrecen un **Single Pane of Glass**, los developer pueden ver como esta todo por detras, un lugar unico y central donde los equipos pueden entender y administrar sus aplicaciones de punta a punta.

Productiza **Golden Paths** para que los equipos apliquen los guardrails practicas de ingenieria organizacionales.

> Es muy dificil que se dejen salir y hacer algo de otra manera, hay que tener una muy buena razon para irse de lo permitido

Si se usa algo nuevo, no se sabe como reaciona, como falla, si es bueno. Eso es un riesgo con un costo altisimo, por eso se trata de que no ocurra.

Evaluacion

Hay que evaluar todos los atributos, ya sean cualitativos o cuantitativos.

Se puede evaluar con:

- Escenarios
- Checklists
- Metricas
- Simulaciones(DSL para evaluar comportamiento)
- Experimentos(software para probar una hipotesis sobre comportamiento)
- Prototipo(version preliminar, podria pasar a ser el producto)
- Modelos matematicos(justifica su empleo en casos de alta critticidad)
- Metodologias basadas en escenarios(SAAM, ATAM, Lightweight ATAM)

  Ejemplo de DSL para simular arquitecturas: Palladio

  Los drivers de negocio son los atributos de calida

Arbol de utilidad

Sirve para enfocar el trabajo a realizar, se arranca con un primer nivel donde cada nodo es un atributo de calidad. Luego, un nivel mas abajo estan los escenarios(segundo nivel), buscan ayudar en la cuantificacion de los atributos. Luego, cada escenario tiene como atributos(tercer nivel):

- Dificultad(de resolver la problematica)
- Importancia

  En el parcial te puede pedir armar un arbol de calidad hasta nivel 2 o mas(no se)

Lightweight ATAM

En vez de tardar 20 dias, se puede hacer en medio dia. Sacan reporte y procesos burocraticos y se enfocan en armar la arquitectura. Como ATAM, se usa para validar un sistemas, que podria ser para validar un sistema que me estan vendiendo o uno que es interno a una empresa que estoy comprando.

Unidad 5: Persistencia

A la hora de elegir una solucion de persistencia, hay que considerar:

- Performance(accesos secuenciales o aleatorios)
- Transformacino de datos
- Tolerancia a fallos

Por otro lado, hay 3 maneras de implementarlo:

- Archivos
- Prevalencia
- Bases de datos

Archivos

La mas sencilla, nuevamente pueden ser:

- Planos(ej. CSV)
- Estructurados(ej. XML)
- Archivos binarios(indexados o serializados)

Los archivos planos son los mas sencillos de implementar y de leer a simple vista. CSV es de acnho variable pues es por filas y se pueden seguir agregando. El principal problema de estos archivos es que debido a su simplicidad tienen muchas restricciones. Por ejemplo, acceso aleatorio, solo se puede ir a un lugar empezando desde el principio. Son utiles cuando leo todo y escribo todo.

Por otro lado, estan los archivos planos de ancho fijo, soluciona el problema de acceso aleatorio. Implica que el ancho es aleatorio es definido arbitrariamente, si tengo muy poco hay que poner padding y si tengo de mas hay que truncar. Sin embargo, restrictivo pues en el caso de tener que agregar un campo hay que cambiar todo. Una solucion que se suele ver es poner caracteres al final reservados para usos futuros.

Serializacion transforma la informacion en un array de bytes, mientras los valores sean serializables. De manera que permite guardar valores en memoria. Sin embargo, trae problemas si cambia la manera en la que se serializa la informacion, generalmente se rompe. No se suele usar para persistencia, pero si para transmitir informacion entre programas.

Prevalencia

Es un patron, los datos se mantienen en memoria en formato nativo, es llevar la serializacion al extremo. Las transacciones se registran, y se hacen **snapshot** a disco, permitiendo levantar de disco a memoria directamente.

Implementaciones:

- Prevayler(Java)
- Madeleine(Ruby)
- Bamboo(.net)

En este caso, no sirve como mecanismo para expandir la capacidad de manipulacion de datos pues la memoria tiene un limite, y si quiero poner todo en memoria no se puede. Sin embargo, si solo quiero preservar estado, esta solucion es completa, flexible y de muy bajo costo.

Ventajas

- No hay transformación, los datos se guardan en el formato que los usa el programa.
- Alto grado de transparencia.
- La performance es muchísimo más alta.

Desventajas

- La aplicación debe poder ser contenida en memoria.
- Interoperabilidad.

Bases de datos

La forma en la que se guarda la informacion es un detalle de implementacion, no es nuestra responsabilidad. Permite acceso concurrente a la informacion, incluso por parte de distintos programas.

Se estan desarrollando hace casi un siglo, lo cual significa que se sabe como fallan, son extremadamente predecibles y se sabe como trabajar con las distintas fallas. No se usa tener logica en la base de datos, hoy en dia los triggers ya son algo malo. Esto se debe a que con triggers se tendria que mantener dos logicas, la de la DB y la de la aplicacion, por lo tanto se tiene una DB tonta(los constraints como `NOT NULL`, etc. si se siguen usando).

Bases de datos relacionales transaccionales(OLTP)

- Organiza datos en tablas(relacionales de filas y columnas)
- Basada en teoria de algebra relacional y conjuntos
- Posee tablas, contraints, keys, etc.
- Se debe normalizar
- Uso para la gestion de datos transaccionales

Bases de datos multidimensionales(OLAP)

Estas se usa para analitica(reporting), casi nada de normalizacion.

- Se almacenan grandes volumenes historicos
- Se guarda desnormalizado(para no hacer joins)
- Se precalculan posibles agregaciones
- Sin bases de datos **append-only**

Se tienen muy pocas tablas muy grandes. Simpre se populan a partir de una base de datos OLTP. Evitando joins aumenta la velocidad de obtencion de informacion. En general se actualiza pasando lo nuevo que haya en la base de datos OLTP periodicamente, usando un proceso batch llamado ETL(Extract, Transform & Load).

```
|-----------|           |------------|
| OLTP      |  -------> | OLAP       |
|-----------|   (ETL)   |------------|
```

Se busca correr en momentos donde la carga del sistema sea poca

Generalmente las DB OLAP tienen un dia de delay, si se quiere algo mas real-time se debe utilizar otra tecnologia.

Bases de datos de objetos

No son tan populares. Guardan objetos complejos(no simples tablas). Lo mas comun es no usar esto y en cambio usar un ORM.

Mapeo objeto relacional(ORM)

Mapea objetos a tablas y tablas a objetos. Por ejemplo, hibernate. Automatizan las operaciones mas comunes(CRUD). Facilitan la programacion, a costa de performance, y complejidad.

Impedancia objeto-relacion

Viene de el detalle que el mundo de los objetos y el mundo relacional no sin isomorfos. Por tanto, se pierde informacion pasando de un lugar a otro.

- Falta de herencia
- Identidad vs. Clave primaria
- Inflexibilidad de las tablas
- FKs vs. Relaciones bidireccionales

Escabilidad de bases de datos relacionales

Las bases de datos relacionales pueden escalar:

- Verticalmente
- Horizontalmente

El escalamiento horizontal, puede hacerse de dos formas distintas:

- Primario - Secundario
- Primario - Primario

  En general, nadie quiere escalar verticalmente, es exponencialmente caro y retorno marginal.

Primario - Secundario

Es el sistema mas comun, el primario se encarga de modificar los datos, mientras que los secundarios se encargan de proveer informacion, son read-only. Tener una mayor escala de operaciones de lectura soluciona la mayoria de los casos pues las operaciones de lectura suelen ser ordenes de magnitud de diferencia con las escrituras. Ademas de mejor performance, ofrece tolerancia a fallas, pues si falla un secundario hay mas, si falla el primario, los nodos secundarios se ponen de acuerdo y un secundario pasa a ser primario.

Primario - Primario

Full-duplex, se pueden escalar tanto las lecturas como las escrituras, pero no es gratis. En este caso todos los nodos deben permanecer sincronizados. A medida que se escala horizontalmente se debe sincronizar aun mas instancias y cada transaccion se debe complir en todos los nodos antes de marcarla como exitosa.

  Uno de los pocos motores de base de datos que lo soporta es Oracle

Teorema CAP

Para todo sistema de almacenamiento de datos, solo se pueden elegir dos de las tres propiedades:

- Consistency(Al clients see current data regardless of updates or delete)
- Availability(The system continues to operate as expected even with node failures)
- Partition tolerance(The system continues to operate as expected despite network or message failures)

La base de datos relacion una vez que empieza a diverger no se puede volver a juntar, hay que elegir una nueva fuente de verdad y tirar la otra. Es decir, son C y A. A continuacion algunas practicas comunes para soportar el aumento continuo de la contidad de lecturas y escrituras en las bases de datos.

Sharding

Teniendo tantos datos que ni el indice de una tabla entra en memoria, performance ya no es ni considerada. Entonces, se divide la base de datos en N copias estructurales iguals y cada una tiene informacion disociada. Ahora pasa a ser responsabilidad de la aplicacion garantizar la consistencia de los datos, esto se logro con:

- Two phase commit(mas de una modificacion en paralelo, si las db commitean, listo)
- Retry until success
- Last write wins
- etc.

Tablas como columnas

Por cada columna se creaba una nueva tabla, luego, cuando habia que buscar informacion se hacia un join con todas las tablas. Esto es parte del origen de las DB no relacionales.

JSOM blobs

Soluciom schema-less encima de MySQL, guardando la informacion como JSON. Permitia hace busquedas sin tener que buscar lo que habia dentro de cada JSON mmediante **index tables** y **entity tables**.

Comienzan entonces a crearse bases de datos dedicadas que **productizan** estos mecanismos y se los abstraen al cliente. Y como todos tenian su propia manera de implementar esto, se las empezo a llamar NoSQL. Hoy en dia esta mal, no son NoSQL sino que son **bases de datos no relacionales**.

Bases de datos no relacionales

Surgen en pare para suplir las limitaciones del modelo relacional.

- Escabilidad
- Distribuidas **por diseno**
- Enormes volumenes de datos
- Esquemas flexibles
- Pueden o no implementar ACID

Columnares

Los datos son guardados ordenados por columan en lugar de por registro, mucho mas rapidas. Mas facil calcular pproyecciones relativas a una o pocas columnas, pero es mas dificil realizar escrituras o consultas de muchas columnas.

Clave/Valor

La información se guarda organizado como un par clave/valor.

- Tiene una alta flexibilidad, no es necesario modificar estructuras para agregar atributos.
- No es necesario lidiar con tipos de datos
- Es complicado hacer búsquedas por más de un campo (AND y OR)

Documentos

Guardan cualquier "documento" arbitrario. Es mas facil guardar estructuras que varian. Sin embargo, no es tan fácil hacer consultas complejas (proyecciones, por ejemplo).

> Lograr un buen diseno es lo mas dificil con estas bases de datos, es facil sin querer terminar con un esquema relacional.

Estas bases implementan **sharding**, pero con escabilidad y (cierta) tolerancia a fallos. Convierten las consultas en funciones programadas de forma funcional. Usan funciones **map-reduce**, consiste en procesar los datos de a grupos, proviene del paradigma funcional, provee escalabilidad y performance al distribuir el computo. Dificulta algunas operaciones como proyecciones. La parte reduce compagina las respuestas, podria ser tan simple como contar la cantidad de elementos.

Detalle interesante, no hay indices, va y procesa todos los elementos, el unico motivo por el cual tiene performance es porque el procesamiento se realiza de forma distribuida. Por fuerza bruta, basicamente.

Ahora bien, como eligo?

Cada opcion difiere mucho de las otras, en terminos de tradeoffs y usos. La gran diferencia va a venir de entender los patrones de acceso que va a haber. A la hora de decidir, lo que importa es el orden de magnitud.

> Si usan una DB NoSQL, tengan un buen motivo, que la escala lo pida

Usar no relacional si se espera un gran volumen de trafico, luego, el tipo a elegir dependera de dos grandes preguntas:

- Patrones de acceso
- Que necesita, CA, CP, AP